Universidad de Alicante

Escuela Politécnica Superior

Grado en Ingeniería en Inteligencia Artificial | Curso 4º (2026/2027)

Robótica Inteligente (33683)

Memoria Técnica de Práctica 1

Introducción a las Herramientas Básicas para Robótica Móvil y Manipulación: ROS 2, Gazebo Harmonic, Nav2, MoveIt 2 y Oxford Spires Dataset

---

Profesor / Tutor:

M.A. Muñoz-Bañón
Dpto. de Física, Ingeniería de Sistemas
y Teoría de la Señal

Equipo de Prácticas:

Santiago Álvarez González (`santvz6`)

Fecha de Evaluación:

Octubre de 2026

Entorno de ejecución: ROS 2 Jazzy Jalisco | Gazebo Harmonic 8.0 | Ubuntu 24.04 LTS (Docker)

Índice

1. Introducción y Estructura Global de la Práctica **1**
1.1. Estructura Modular de la Práctica 1

2. Bloque 4 – Manipulación Robótica (UR3e y MoveIt 2) **1**
2.1. Contexto y Arquitectura de los Tres Pilares 1
2.2. Despliegue y Control Interactivo en RViz2 2
2.2.1. Operación en el Espacio de Trabajo 2
2.3. Introspección del Middleware ROS 2 (Sección 6.1) 2
2.3.1. Mapeo Detallado de Nodos e Interfaces 2
2.3.2. Grafo Computacional Dinámico (`rqt_graph`) 3
2.4. Explicación del Flujo de Información y Cinemática 3
2.5. Análisis e Interpretación del Árbol de Transformaciones (TF2) 4
2.6. Tarea de Programación Opcional: Control en Python (Sección 6.2) 4

1 Introducción y Estructura Global de la Práctica

La presente memoria técnica documenta el desarrollo integral y el análisis exhaustivo de la **Práctica 1 de Robótica Inteligente**. El objetivo central consiste en capacitar al alumnado en las herramientas de vanguardia para investigación e industria en robótica: el middleware distribuido **ROS 2 (Jazzy Jalisco)**, el simulador dinámico **Gazebo Harmonic** y los frameworks avanzados de navegación autónoma (**Nav2**), odometría LiDAR-inercial (**RKO-LIO**) y manipulación robótica industrial (**MoveIt 2**).

Toda la suite de desarrollo opera encapsulada sobre un contenedor persistente de **Docker** con aceleración gráfica por hardware (GPU NVIDIA dedicada, soporte PRIME Offload y paso directo X11), eliminando conflictos de dependencias y asegurando la reproducibilidad íntegra de los experimentos.

1.1 Estructura Modular de la Práctica

De acuerdo con las directrices de la guía docente, la práctica se estructura en cuatro bloques secuenciales:

- **Bloque 1 – Fundamentos de ROS 2:** Dominio de la arquitectura orientada a grafos de nodos, canales asíncronos (tópicos), llamadas síncronas (servicios) y tareas continuas con retroalimentación (acciones).
- **Bloque 2 – Mapeo y Navegación Autónoma (Nav2):** Generación de mapas de ocupación 2D mediante *SLAM Toolbox* y control de plataforma móvil diferencial *TurtleBot 4* empleando filtros de partículas (AMCL) y controladores predictivos MPPI. *[Desarrollo documentado en anexo correspondiente].*
- **Bloque 3 – Odometría LiDAR-Inercial (LIO) con Datos Reales:** Procesamiento sensorial denso sobre secuencias reales del *Oxford Spires Dataset* (Keble College) mediante el pipeline *RKO-LIO. [Desarrollo documentado en anexo correspondiente].*
- **Bloque 4 – Manipulación Robótica (MoveIt 2):** Control cinemático cartesiano y articular del brazo industrial *Universal Robots UR3e* (6 DOF) en simulación dinámica y planificación libre de colisiones.

2 Bloque 4 – Manipulación Robótica (UR3e y MoveIt 2)

2.1 Contexto y Arquitectura de los Tres Pilares

La manipulación robótica aborda el gobierno de cadenas cinemáticas abiertas articuladas en el espacio tridimensional ($\mathbb{R}^3 \times \mathrm{SO}(3)$). A diferencia del desplazamiento móvil restringido al plano bidimensional SE(2), un brazo articulado debe gestionar acoplamientos no lineales de inercia, singularidades cinemáticas del jacobiano y la resolución de geometrías de colisión complejas.

El sistema desplegado integra el cobot de 6 grados de libertad **Universal Robots UR3e** sustentado sobre tres componentes tecnológicos que interactúan de forma desacoplada:

1. **Gazebo Harmonic (`gz-sim`):** Motor de física de cuerpos rígidos (*DART Physics Engine*). Modela la gravedad, inercias de eslabones, restricciones de articulación (*revolute joints*) y dinámica de contactos.
2. **ros2_control (`controller_manager`):** Capa de abstracción de hardware en tiempo real. Gestiona el ciclo de vida del `joint_state_broadcaster` (lectura de encoders) y del `scaled_joint_trajectory_controller` (seguimiento de trayectorias articulares con escalado dinámico de velocidad ante sobrecargas).

3. **MoveIt 2 (`move_group`):** Núcleo de computación cinemática y planificación espacial. Gestiona el estado de la escena (*Planning Scene*), resuelve la cinemática inversa analítica/numérica y utiliza algoritmos de muestreo probabilístico (**OMPL**) para trazar caminos libres de colisiones.

2.2 Despliegue y Control Interactivo en RViz2

El ecosistema completo se inicializa mediante un único script de *launch* optimizado:

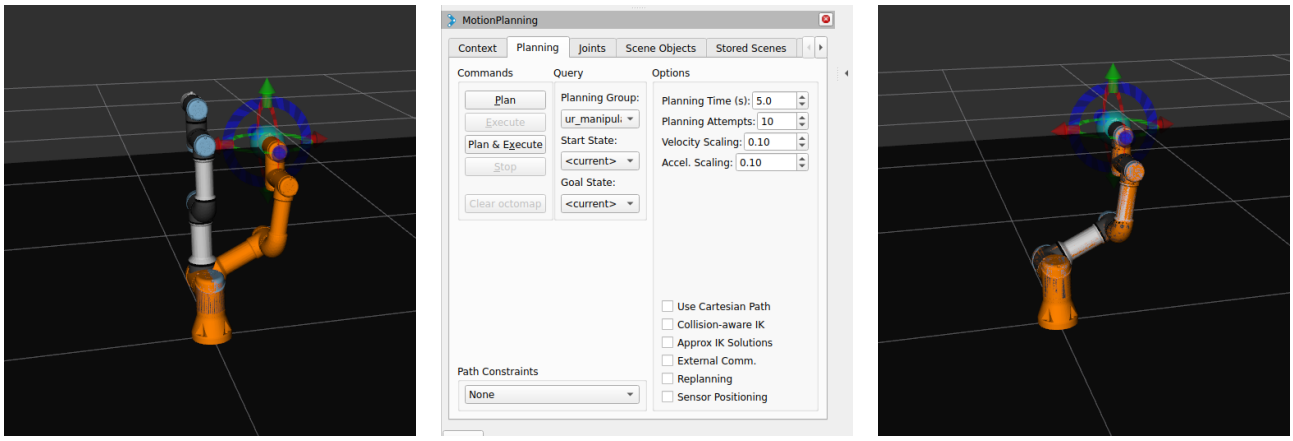
```
ros2 launch ur_simulation_gz ur_sim_moveit.launch.py ur_type:=ur3e
```

Este comando arranca simultáneamente el simulador Gazebo Harmonic y la interfaz de visualización 3D **RViz2** configurada con el plugin *MotionPlanning* y el árbol URDF del robot.

2.2.1 Operación en el Espacio de Trabajo

El control interactivo se realiza gobernando el efector final (`tool0`):

1. **Marcador Interactivo:** RViz dibuja un manipulador esférico con anillos ortogonales de rotación y flechas de traslación cartesianas (X, Y, Z) sobre el extremo del robot.
2. **Estado Objetivo (*Goal State*):** Al arrastrar el marcador con el ratón, se calcula en tiempo real la cinemática inversa y se proyecta la postura destino mediante un robot semitransparente de color naranja.
3. **Planificación y Ejecución (*Plan and Execute*):** En el panel *MotionPlanning*, la pestaña *Planning* ejecuta el planificador OMPL (algoritmo RRTConnect) y transmite los splines interpolados a `ros2_control`, haciendo que el brazo físico en Gazebo ejecute la trayectoria de forma suave.



(a) Arrastre interactivo (*Goal State*). (b) Panel *MotionPlanning*. (c) Trayectoria ejecutada con éxito.

Figura 1: Secuencia de interacción manual en RViz2: fijación de meta cartesiana 3D mediante marcador interactivo, cálculo de trayectoria OMPL y ejecución dinámica en Gazebo.

2.3 Introspección del Middleware ROS 2 (Sección 6.1)

Conforme a los requerimientos de la Sección 6.1 del guion, se procedió a interrogar al middleware utilizando herramientas CLI de introspección activa. Se detectaron **19 nodos funcionales** coordinando la simulación.

2.3.1 Mapeo Detallado de Nodos e Interfaces

La Tabla 1 resume las entradas, salidas, servicios y acciones de los nodos medulares del pipeline.

Nodo Activo	Tópicos Suscritos	Tópicos Publicados	Servicios / Acciones
`/move_group`	`/joint_states`, `/clock`, `/trajectory_execution_event`	`/display_planned_path`, `/monitored_planning_scene`	Acciones: `/move_action`, `/execute_trajectory`. Srv: `/compute_cartesian_path`
`/controller_manager`	`/clock`, `/robot_description`	`/controller_manager/activity`, métricas de lazo	Srv: `/switch_controller`, `/load_controller`
`/joint_state_broadcaster`	`/clock`	`/joint_states`, `/dynamic_joint_states`	Srv: Parámetros del broadcaster
`/scaled_joint_trajectory_controller`	`/clock`, consignas de trayectoria	`/controller_state`, eventos de transición	Acción: `/scaled_joint_trajectory_controller/follow_joint_trajectory`
`/robot_state_publisher`	`/joint_states`, `/clock`	`/tf`, `/tf_static`, `/robot_description`	Srv: Introspección URDF
`/rviz2_moveit`	`/display_planned_path`, `/tf`, `/tf_static`	Feedback de marcadores, `/goal_pose`	Cliente de acción: `/move_action`

Tabla 1: Mapeo de interfaces de los componentes principales del Bloque 4.

2.3.2 Grafo Computacional Dinámico (`rqt_graph`)

El análisis gráfico ejecutado con `ros2 run rqt_graph rqt_graph` (Figura 2) confirma la arquitectura desacoplada: `joint_state_broadcaster` actúa como el puente sensorial alimentando a `robot_state_publisher` y a `move_group`, mientras que la ejecución de comandos se delega al controlador articular de `ros2_control`.

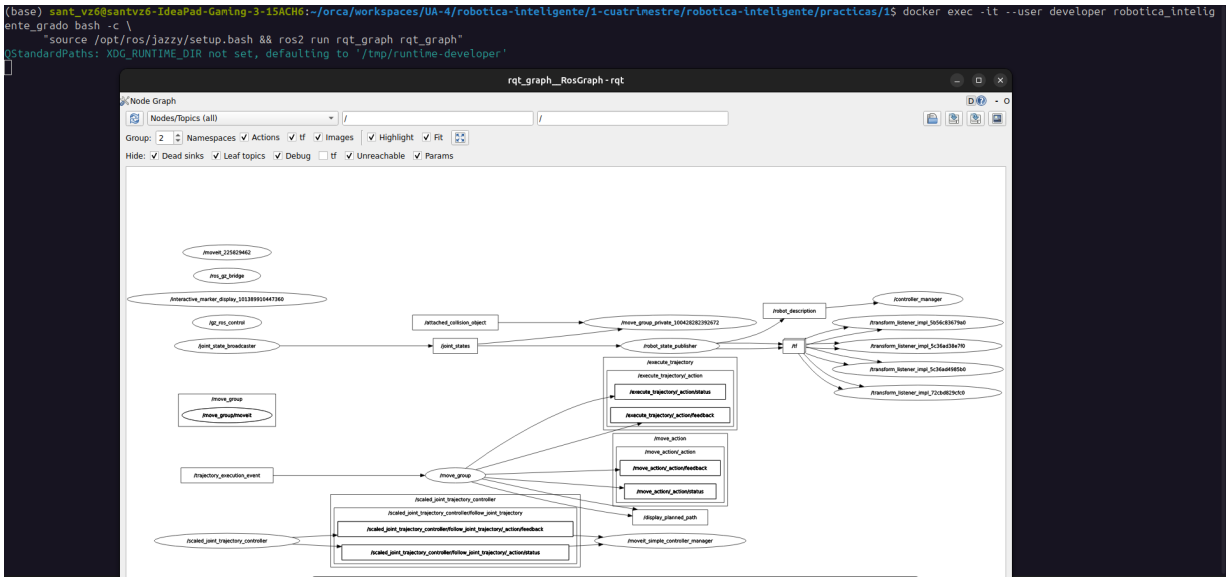


Figura 2: Grafo dinámico de nodos y tópicos del sistema capturado en vivo mediante `rqt_graph`.

2.4 Explicación del Flujo de Información y Cinemática

Para comprender cómo funciona el sistema en conjunto, analizamos el recorrido de la información desde que solicitamos una postura hasta que el robot se mueve en el simulador:

1. **Definición de la meta (Pose objetivo):**
   Se indica la posición (X, Y, Z) y la orientación deseada para el extremo del robot (`tool0`). Esta orden puede enviarse de forma manual arrastrando el marcador 3D en RViz2 o mediante comandos desde la terminal utilizando el script en Python.

2. **Cálculo de cinemática inversa y planificación (MoveIt 2):**
   El nodo `move_group` procesa la petición realizando dos pasos fundamentales:
   - **Cinemática inversa (IK):** Calcula los ángulos necesarios en cada una de las 6 articulaciones del brazo $(\theta_1, \ldots, \theta_6)$ para situar el extremo en la coordenada espacial solicitada.
   - **Planificación de trayectoria:** Con los ángulos objetivo, el planificador (OMPL) busca un camino libre de colisiones entre la postura actual y la meta, generando una secuencia de puntos suave que respeta los límites de velocidad y aceleración del brazo.

3. **Ejecución del movimiento (`ros2_control`):**
   Una vez calculada la trayectoria, MoveIt la transmite al controlador articular de trayectoria mediante la acción de ROS 2 `FollowJointTrajectory`. El nodo `scaled_joint_trajectory_controller` guía a los motores para recorrer la ruta en el tiempo previsto.

4. **Simulación física y realimentación sensorial (Gazebo y TF2):**
   Gazebo aplica el movimiento a las articulaciones físicas del robot en el entorno virtual. En paralelo, los sensores del simulador leen la posición real de cada articulación y la publican en el tópico `/joint_states`. A partir de estas lecturas, el nodo `robot_state_publisher` actualiza el árbol de transformaciones geométricas (`/tf`), permitiendo que RViz2 refleje en tiempo real la postura exacta del robot.

2.5 Análisis e Interpretación del Árbol de Transformaciones (TF2)

La comprensión espacial de la cadena cinemática se verificó mediante la herramienta `ros2 run tf2_tools view_frames` (Figura 3).



(a) Extracción de *frames* con `view_frames`.



(b) Inspección en vivo con `echo /tf --once`.

Figura 3: Introspección del árbol espacial de transformaciones geométricas (TF2) en consola.

El árbol jerárquico se compone de **14 marcos de referencia** encadenados:

$$\text{world} \longrightarrow \text{base_link} \longrightarrow \text{base_link_inertia} \longrightarrow \text{shoulder_link} \longrightarrow \text{upper_arm_link}$$
$$\longrightarrow \text{forearm_link} \longrightarrow \text{wrist_1_link} \longrightarrow \text{wrist_2_link} \longrightarrow \text{wrist_3_link} \longrightarrow \text{flange} \longrightarrow \text{tool0}$$

- **Transformaciones Estáticas (`/tf_static`):** Invariantes en el tiempo, tales como la fijación del robot al suelo (`world ->base_link`), la inercia interna de la peana y la distancia fija entre la brida mecánica y el centro de herramienta (`flange ->tool0`).
- **Transformaciones Dinámicas (`/tf`):** Publicadas a una frecuencia nominal de **20.16 Hz** por `robot_state_publisher`. Cada enlace móvil transmite un vector de traslación constante (longitud del eslabón) y un cuaternión de rotación unitario variable que refleja en tiempo real el ángulo del servomotor.

2.6 Tarea de Programación Opcional: Control en Python (Sección 6.2)

Para aspirar a la máxima calificación según la Sección 6.2 del guion, se programó y validó el script en Python `ur3e_send_pose.py` en el espacio de trabajo persistente (`~/ros2_ws/p1_scripts/`). Este script prescinde enteramente de la interfaz gráfica RViz2, conectándose como cliente directo a las acciones de ROS 2.

Se validaron tres modos de control en la consola (Figura 4):

1. **Prueba A – Modo Articular Directo (`--mode joint`):**
   Envío directo de la configuración articular $\boldsymbol{\theta} = [0{,}0, -1{,}57, 1{,}57, -1{,}57, -1{,}57, 0{,}0]$ rad al nodo `scaled_joint_trajectory_controller`. El manipulador flexiona el codo en ángulo recto con perfil suave interpolado en 4.0 segundos.

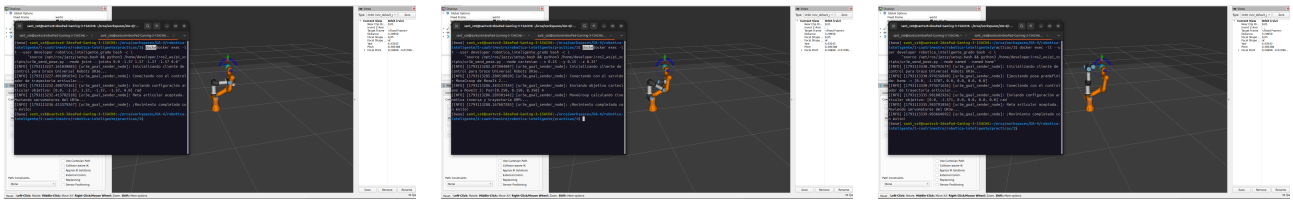
   ```
   python3 ur3e_send_pose.py --mode joint --joints 0.0 -1.57 1.57 -1.57 -1.57 0.0
   ```

2. **Prueba B – Modo Cinemático Cartesiano 3D (`--mode cartesian`):** Fijación de coordenada espacial ($X = 0{,}25\,\text{m}, Y = 0{,}15\,\text{m}, Z = 0{,}35\,\text{m}$) con orientación vertical (pitch $= \pi$). La orden viaja al servidor de acción `/move_action` de MoveIt 2, el cual calcula cinemática inversa y trayectoria OMPL sin colisiones antes de ejecutar.

   ```
   python3 ur3e_send_pose.py --mode cartesian --x 0.25 --y 0.15 --z 0.35
   ```

3. **Prueba C – Poses Semánticas SRDF (`--mode named`):** Carga de posturas preestablecidas de fábrica (`home`, `up`, `test_configuration`) definidas en el archivo semántico SRDF de Universal Robots.

   ```
   python3 ur3e_send_pose.py --mode named --named home
   ```



(a) Prueba A: Control Articular. (b) Prueba B: Meta Cartesiana 3D. (c) Prueba C: Pose Semántica *home*.

Figura 4: Validación experimental de la tarea de programación opcional (Sección 6.2): ejecución de comandos desde la terminal y respuesta síncrona del robot en RViz2 y Gazebo.